DATA FLOW CFD: 95%

DIAGNOSIS

← NEW ANALYSIS

OBSERVED SYMPTOM

COPY

The AST pre-analysis fails with an error message indicating that the 'Parser class with .init()' could not be located in 'web-tree-sitter'. This prevents the core analysis engine from utilizing static analysis information.

REPRODUCTION PATH

1. Open the application in a browser.
2. Navigate to Step 2 (Inject Architecture).
3. Provide some code files (e.g., upload a directory or paste files).
4. Click the 'Execute Engine' button.
5. Observe the console output. An error message '[AST] Analysis failed, proceeding without: web-tree-sitter: could not locate Parser class with .init() in any known export shape.' will appear.

CONFIDENCE EVIDENCE

COPY

VERIFIED:

- Console log: 'web-tree-sitter: could not locate Parser class with .init() in any known export shape.' This indicates that the object assigned to `TreeSitter` does not have the expected `init` method, implying an incorrect module import.
- UnravelAI/unravel-v3/src/core/ast-engine-ts.js L37: `TreeSitter = (await import('web-tree-sitter')).default;` explicitly attempts to access the '.default' export.
- The `web-tree-sitter` library typically exposes its main class/object directly as the module export, rather than as a 'default' export. Therefore, accessing `.default` will result in an incorrect or `undefined` object being assigned to `TreeSitter`.

CONCEPT TO LEARN

COPY

MODULE IMPORT/EXPORT

Understanding Module Export Shapes (Default vs. Module Object)

Incorrectly importing modules can lead to `undefined` objects, runtime errors, and broken application functionality. It's a common source of 'works on my machine but not in build' issues because different build tools (like Vite or Webpack) might interpret module exports slightly differently if the import statement isn't precise.

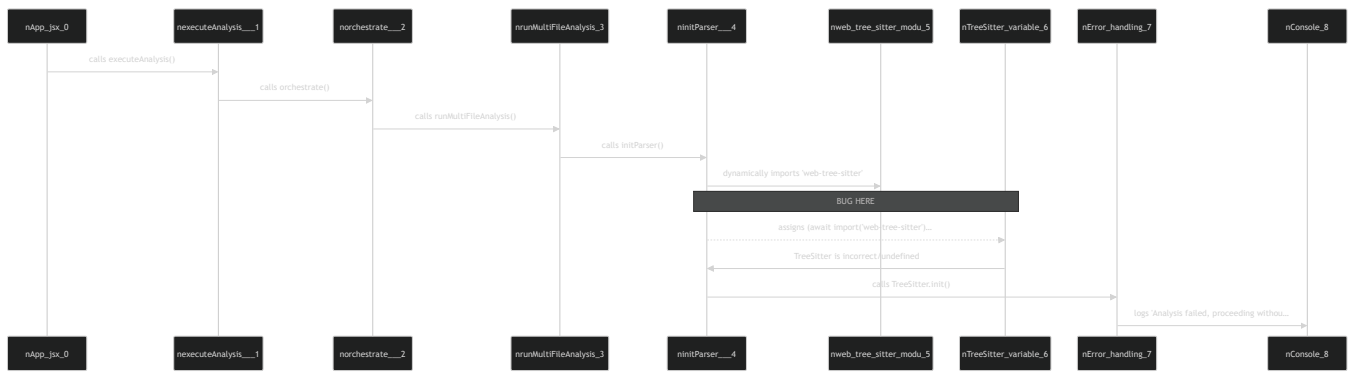
> **PATTERN TO AVOID:**
> Do not blindly assume a 'default' export when importing modules, especially for complex libraries like WebAssembly-based ones. Always verify the exact export structure or try importing the module object directly if the '.default' approach fails.

💡 *Imagine you order a special chai from your favorite roadside stall, 'Chai pe Charcha'. You ask for 'their special (default) chai'. But actually, their special chai is just called 'Chai pe Charcha special' and is handed to you directly, not wrapped inside a box labeled 'default'. You were looking for a 'default' label on your chai, but the chai *itself* was the special. Because you insisted on looking for the 'default' box, you didn't get your chai. The code was looking for the 'default' export when 'web-tree-sitter' directly exports its main functionality.*

💡 REAL WORLD ANALOGY

COPY

*"Imagine you order a special chai from your favorite roadside stall, 'Chai pe Charcha'. You ask for 'their special (default) chai'. But actually, their special chai is just called 'Chai pe Charcha special' and is handed to you directly, not wrapped inside a box labeled 'default'. You were looking for a 'default' label on your chai, but the chai *itself* was the special. Because you insisted on looking for the 'default' box, you didn't get your chai. The code was looking for the 'default' export when 'web-tree-sitter' directly exports its main functionality."*



EXECUTION SEQUENCE — 🦎 MARKS WHERE THE BUG MANIFESTS

INVARIANT VIOLATIONS

COPY

- The `web-tree-sitter` library's main module object, containing `init()` and `Language.load()`, must be correctly assigned to the `TreeSitter` variable in `ast-engine-ts.js` (L37).
- The `TreeSitter` object must always be initialized by calling its static `init()` method before attempting to load language parsers or create parser instances.
- The `web-tree-sitter` package must remain excluded from Vite's dependency optimization (`optimizeDeps.exclude`) in `vite.config.js` to ensure proper WASM asset loading and prevent pre-bundling conflicts.

TECHNICAL ROOT CAUSE

COPY

The root cause is an incorrect module import statement for the `web-tree-sitter` library in `UnravelAI/unravel-v3/src/core/ast-engine-ts.js` at L37. The code attempts to import `web-tree-sitter` and specifically access its `.default` property (`(await import('web-tree-sitter')).default`). However, the `web-tree-sitter` library does not provide its main `TreeSitter` class/object as a default export. Instead, it exports it directly as the module object itself. This mismatch causes `TreeSitter` to be assigned an incorrect or undefined value, leading to the subsequent failure when `TreeSitter.init()` is invoked.

```
Location: UnravelAI/unravel-v3/src/core/ast-engine-ts.js L37
```

CRASH SITE

COPY

WHERE THE FAILURE BECAME VISIBLE ↓

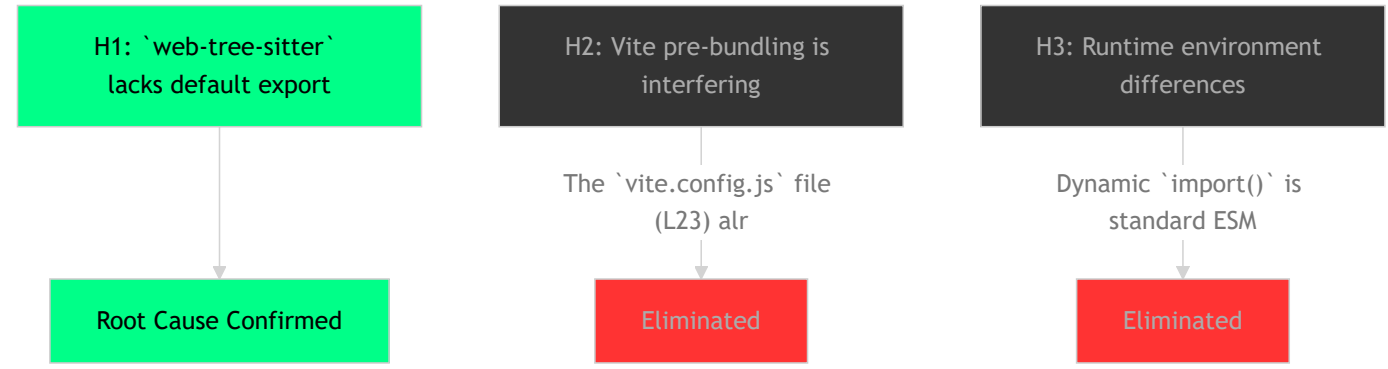
initParser() L42 — The `TreeSitter.init()` call fails because `TreeSitter` is not the expected object with an `init` method.

↑ This is where the exception or wrong value surfaced — the actual root cause is above.

ALTERNATIVE HYPOTHESES

- H1: `web-tree-sitter` does not have a `default` export for its main `TreeSitter` object, causing the import to fail.
- H2: Vite's pre-bundling for `web-tree-sitter` is interfering despite the `optimizeDeps.exclude` configuration.

- H3: A subtle runtime environment difference (e.g., Node.js vs. Browser ESM behavior) is causing `web-tree-sitter` to be imported incorrectly.



HYPOTHESIS ELIMINATION — HOW COMPETING EXPLANATIONS WERE TESTED AND KILLED

DETERMINISTIC AI FIX PROMPT

COPY PROMPT

Paste this directly into Cursor / Bolt / Copilot to apply the fix:

```
The user has reported a bug where the AST analysis is failing. The root cause is an incorrect module import for 'web-tree-sitter'. The 'TreeSitter' object is being imported using '.default', but 'web-tree-sitter' does not expose its main object as a default export. This results in 'TreeSitter' being undefined or an incorrect object, causing 'TreeSitter.init()' to fail. The fix involves removing '.default' from the import statement to correctly capture the module object.

Here's the minimal fix:

```javascript
// FILE: UnravelAI/unravel-v3/src/core/ast-engine-ts.js
// ...
        if (!TreeSitter) {
            TreeSitter = await import('web-tree-sitter');
        }
// ...
```

Explain why this fix works by detailing how 'web-tree-sitter' typically exports its main object and why accessing '.default' was incorrect. Ensure the explanation is clear and easy to understand for a developer confused by this specific bug. Describe the full trace from the original incorrect import to the fix. Provide the output in simple plain English, like explaining to a 15-year-old.
```

MINIMAL CODE FIX

COPY

FILE: UNRAVELAI/UNRAVEL-V3/SRC/CORE/AST-ENGINE-TS.JS L37

```javascript
// FILE: UnravelAI/unravel-v3/src/core/ast-engine-ts.js
// ...
        if (!TreeSitter) {
            TreeSitter = await import('web-tree-sitter');
        }
// ...
```

WHY THIS WORKS

This fix works because it correctly imports the `web-tree-sitter` library. The `web-tree-sitter` package, like many ESM modules, exports its main functionality (the `TreeSitter` class/object) directly as the module itself, rather than wrapping it inside a `default` export. By removing `.default` from `TreeSitter = (await import('web-tree-sitter')).default;` and changing it to `TreeSitter = await import('web-tree-sitter');`, the entire module object is directly assigned to the `TreeSitter` variable. This module object contains the static `init` method and `Language.load` methods that are required for the parser to initialize correctly. With `TreeSitter` now holding the correct object, `TreeSitter.init()` can be called successfully, and the AST pre-analysis pipeline can proceed as intended.

ACTION CENTER

CREATE GITHUB ISSUE

COPY FIX CLI

Create Issue opens a pre-filled GitHub issue in a new tab. You review before submitting.

NEW ANALYSIS